

IMAGE FORMATION

Lecturer: Dr. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

Objectives

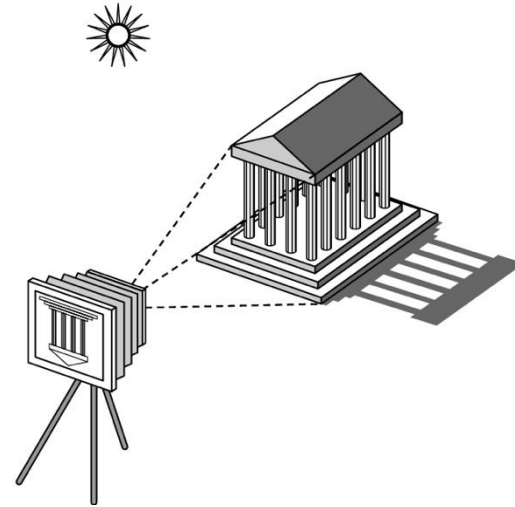
- Fundamental imaging notions
- Physical basis for image formation
 - Light
 - Color
 - Perception
- Synthetic camera model
- Other models

Image Formation

- In computer graphics, we generate 2D images using a process analogous to how images are formed by physical imaging systems
- similar to
 - Cameras
 - Microscopes
 - Telescopes
 - Human visual system

Elements of Image Formation

- Objects
- Viewer
- Light source(s)



- Attributes : that govern how light interacts with the materials in the scene
- Note the **independence** of the objects, the viewer, and the light source(s)

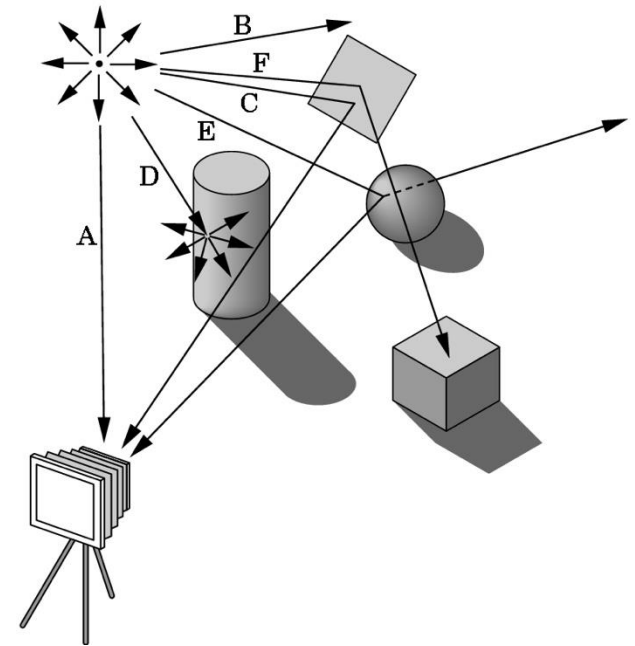
Light

- *Light* is the part of the electromagnetic spectrum that causes a reaction in our visual systems
- Generally these are wavelengths in the range of about 350-750 nm (nanometers)
- Long wavelengths appear as **reds**
- and short wavelengths as **blues**



Ray Tracing and Geometric Optics

- One way to form an image is to follow rays of light from a point source finding which rays enter the lens of the camera.
- However, each ray of light may have multiple interactions with objects before being absorbed or going to infinity.

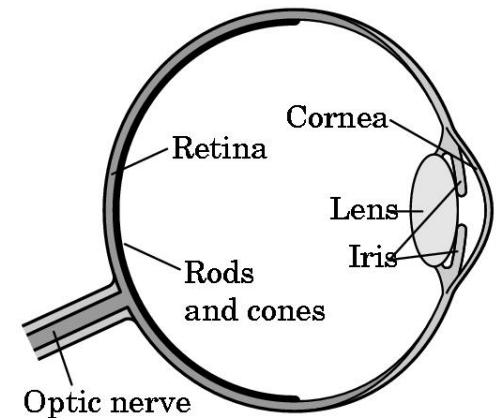


Luminance and Color Images

- Luminance (Black and White / Gray Level) Image
 - Monochromatic
 - Values are gray levels
 - Analogous to working with black and white film or television
- Color Image
 - Has perceptual attributes of hue, saturation, and lightness
 - Do we have to match every frequency in visible spectrum? **No!**

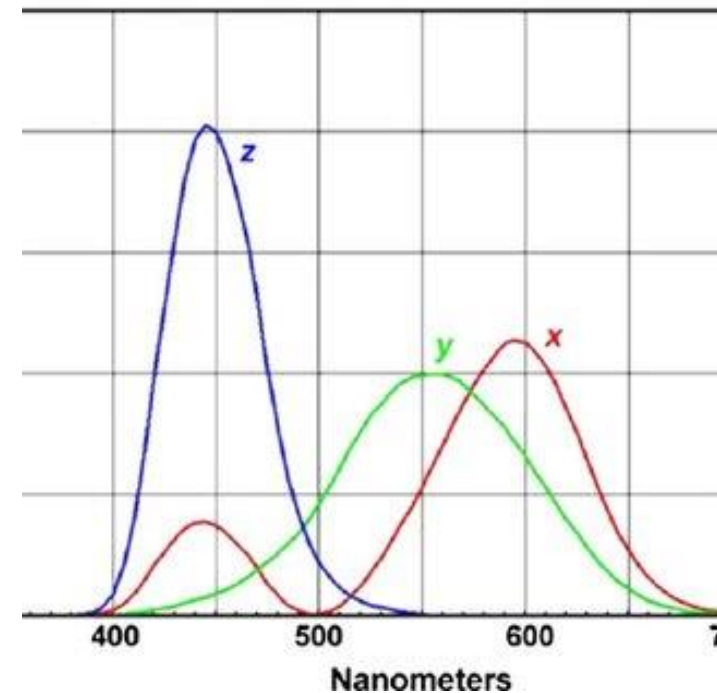
Three-Color Theory

- Human visual system has two types of sensors
 - Rods: monochromatic, night vision
 - Cones
 - Color sensitive
 - Three types of cones
 - Only three values (**the tristimulus values**) are sent to the brain
- Need only match these **tristimulus** values
 - Need only three *primary* colors



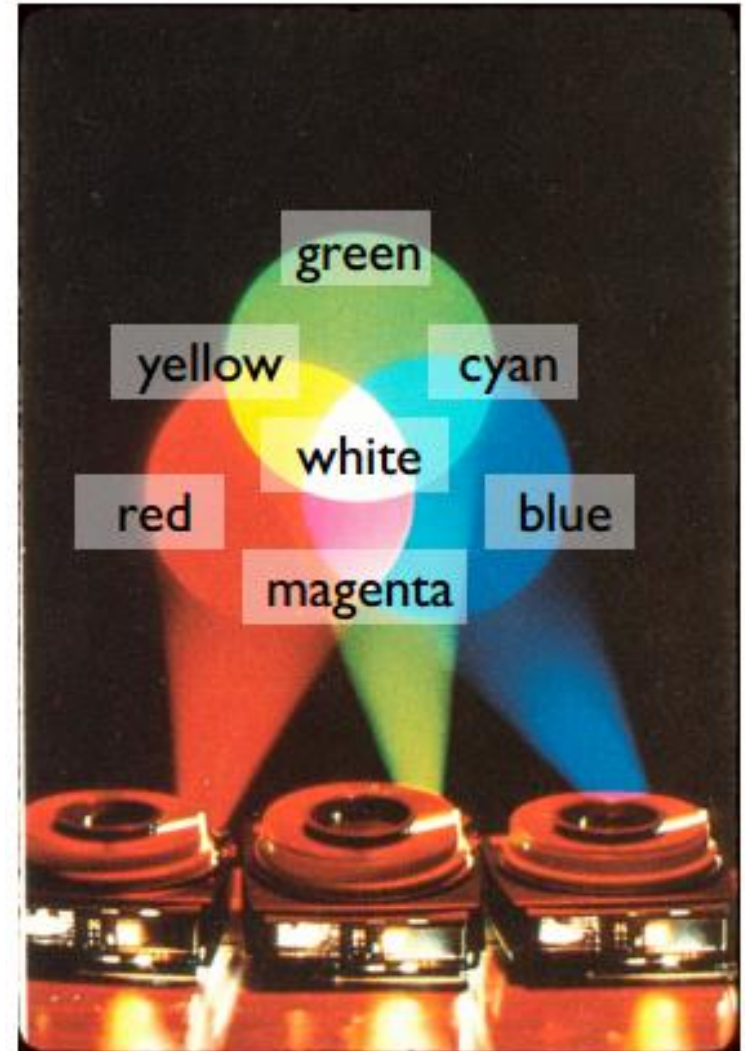
Three-Color Theory

- Human visual system has two types of sensors
 - Rods: monochromatic, night vision
 - Cones
 - Color sensitive
 - Three types of cones
 - Only three values (**the tristimulus values**) are sent to the brain
- Need only match these **tristimulus** values
 - Need only three *primary* colors



Additive Color

- Form a color by adding amounts of three primaries
 - CRTs, LEDs, projection systems..
- Primaries are Red (R), Green (G), Blue (B)
- white is the "additive" combination of all primary colored lights,
- while **black** is the absence of light.



Subtractive Color

- Form a color by filtering white light with Cyan (C), Magenta (M), and Yellow (Y) filters
- The ink reduces the light that would otherwise be reflected.
- Such a model is called subtractive because inks "subtract" brightness from white.
- Printing
- Negative film

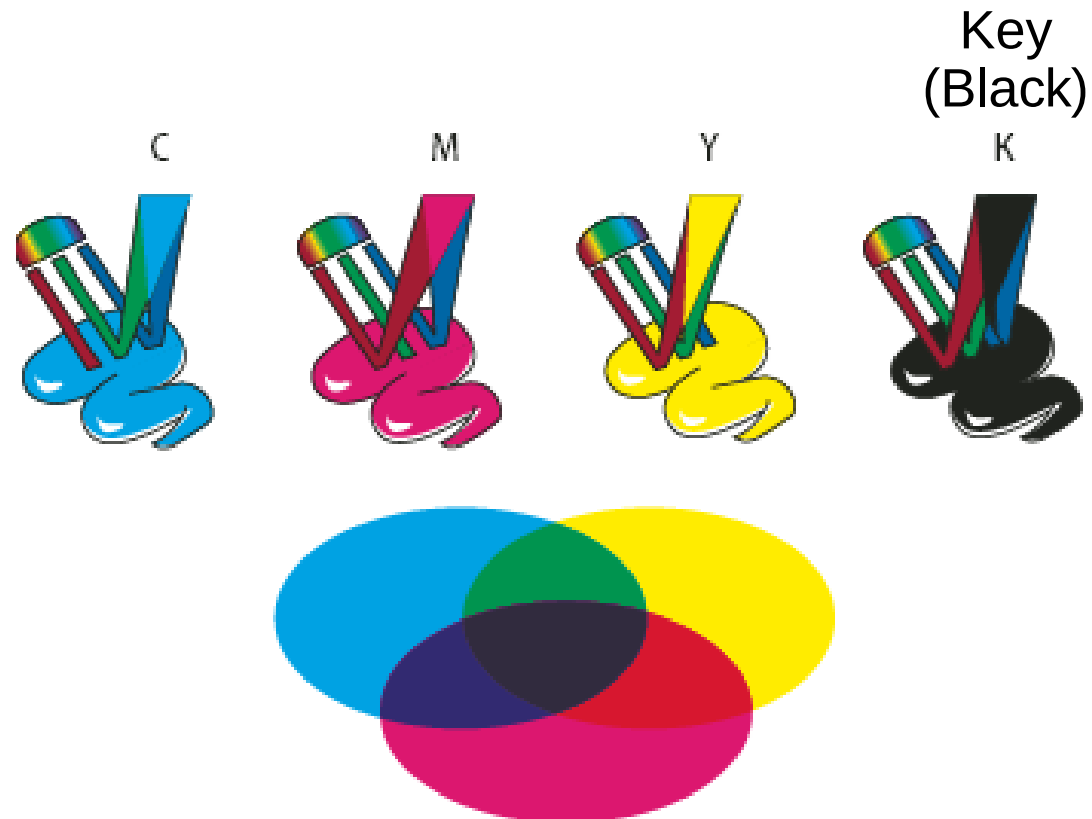
CMYK
Subtractive
Color



CYAN MAGENTA YELLOW

Subtractive Color

- To save money on ink, and to produce deeper black tones, unsaturated and dark colors are produced by using **black** ink instead of the combination of cyan, magenta and yellow



Color Displays

- CRT: phosphor dot pattern to produce finely interleaved color images

- Shadow Mask method
- System has 3 electron guns (R, G, B)
- Each Gun is controlled by Frame buffer

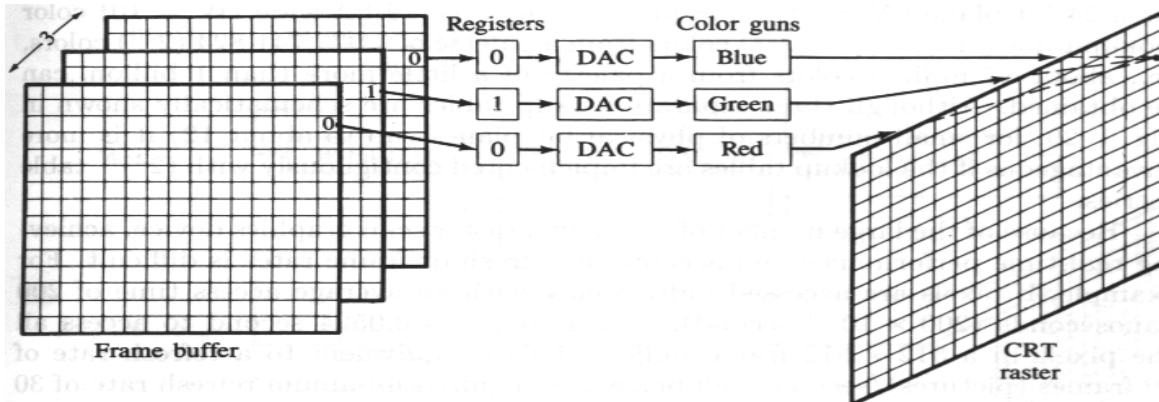
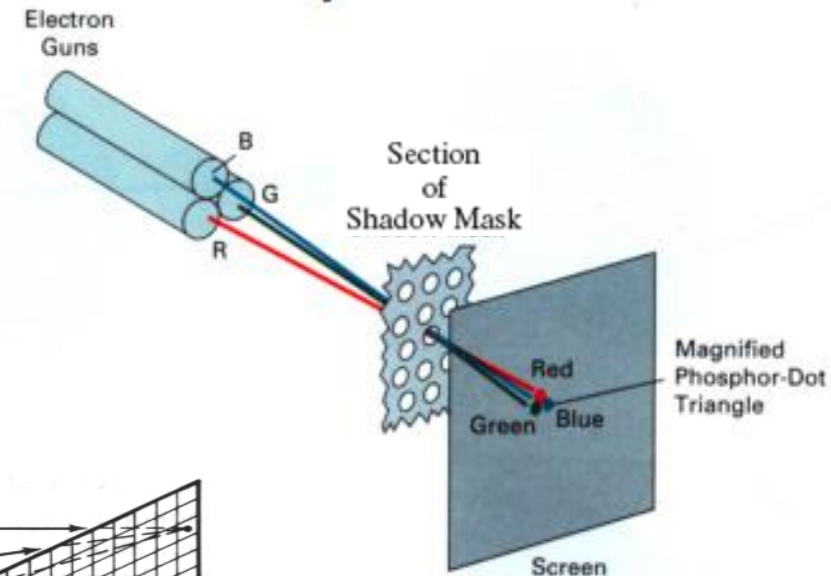
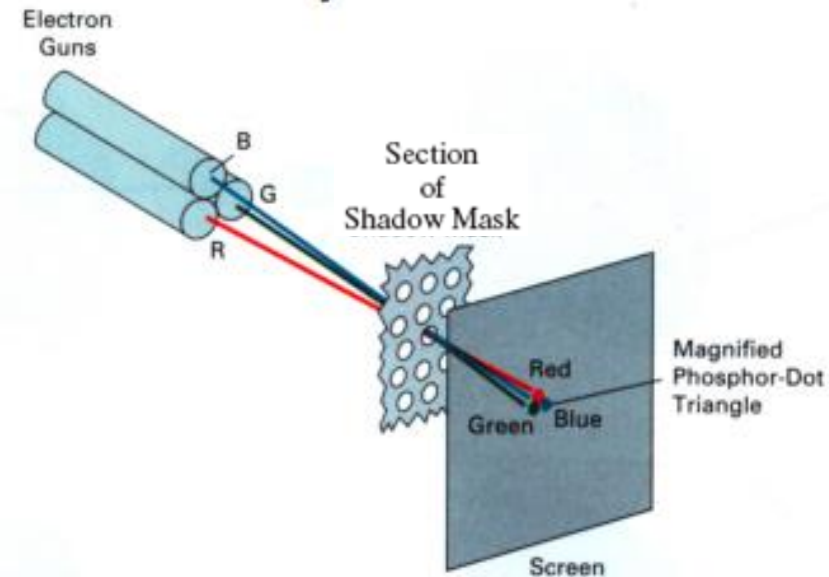


Figure 1-27 Simple color frame buffer.

Color Displays

- CRT: phosphor dot pattern to produce finely interleaved color images



Color Displays

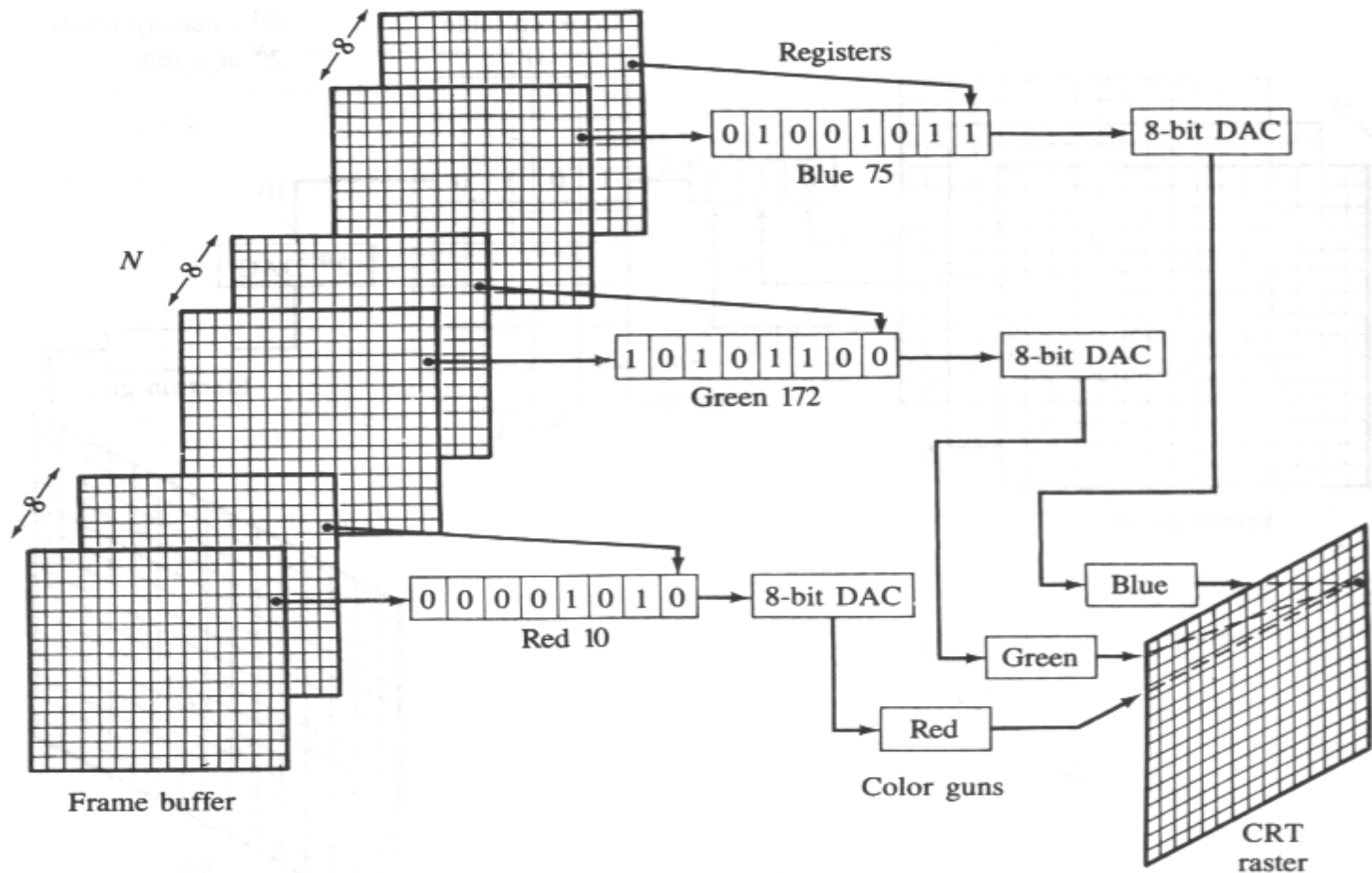


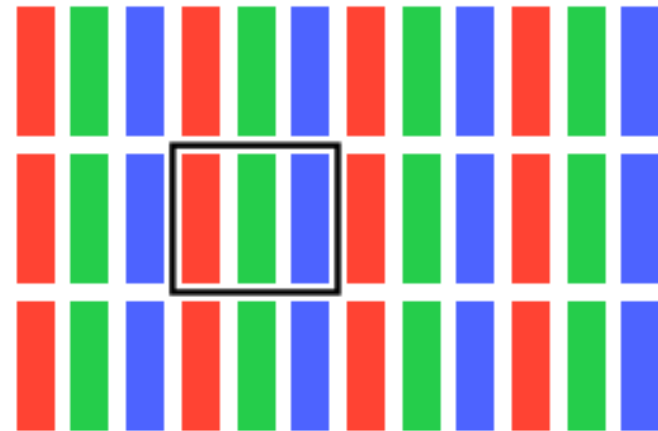
Figure 1-28 A 24-bit-plane color frame buffer.

Color Displays

Today, flat-screen technologies dominate.

- Flat-panel monitors are inherently raster based.
 - Although there are multiple technologies available,
 - including light-emitting diodes (LEDs), liquid-crystal displays (LCDs), and plasma panels
- all use a **two-dimensional grid** to address individual light-emitting elements.

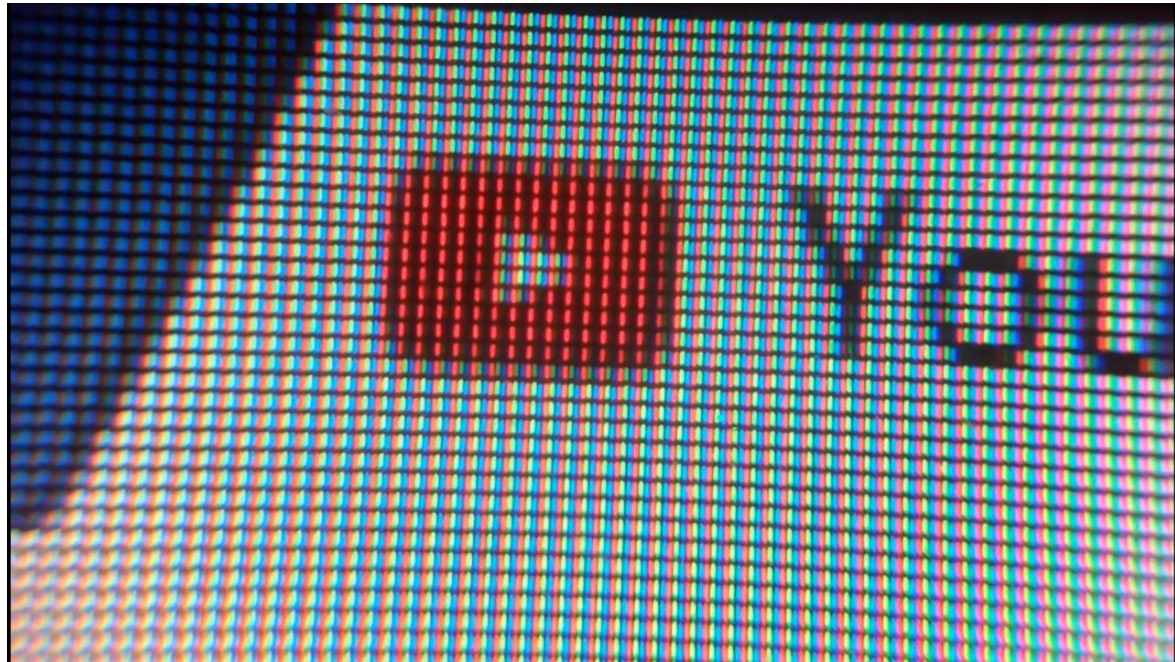
LCD: interleaved R,G,B pixels



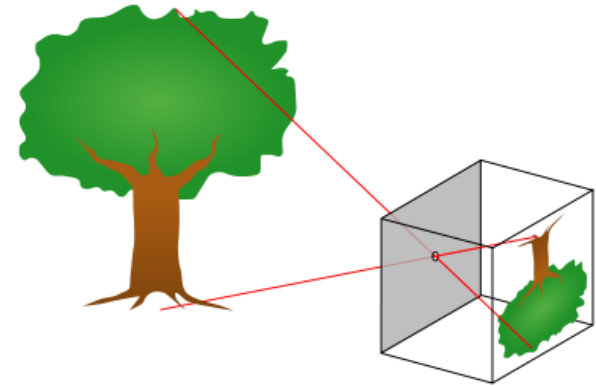
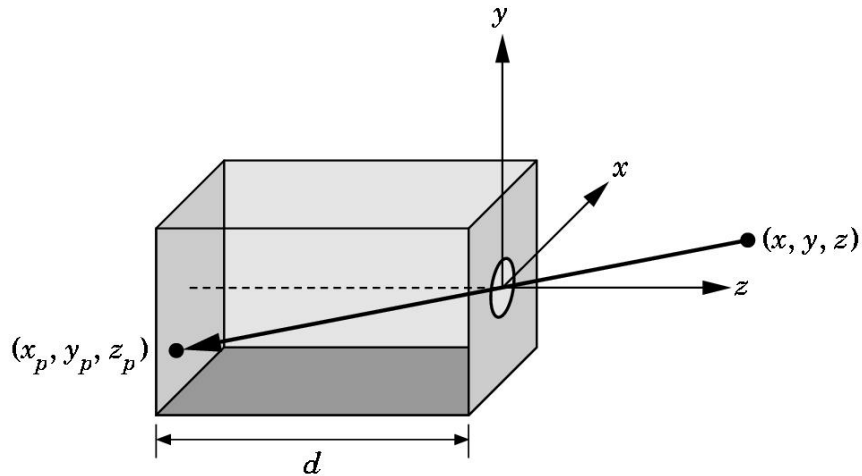
Color Displays

Today, while CRTs are still in use, flat-screen Technologies dominate.

- Flat-panel monitors are inherently raster based.
 - Although there are multiple technologies available,
 - including light-emitting diodes (LEDs), liquid-crystal displays (LCDs), and plasma panels,
- all use a **two-dimensional grid** to address individual light-emitting elements.

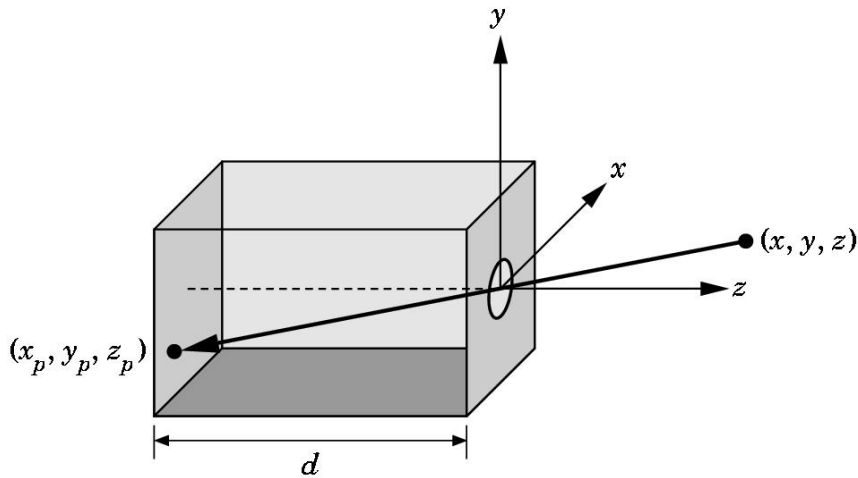


Pinhole Camera



- The pinhole camera is a simple example of an imaging system that lets us understand the functioning of cameras and of other optical imagers with a simple geometric model .
- We emulate it to build a model of image formation.
- A pinhole camera is a box with a small hole in the center of one side of the box; the film is placed inside the box on the side opposite the pinhole.

Pinhole Camera



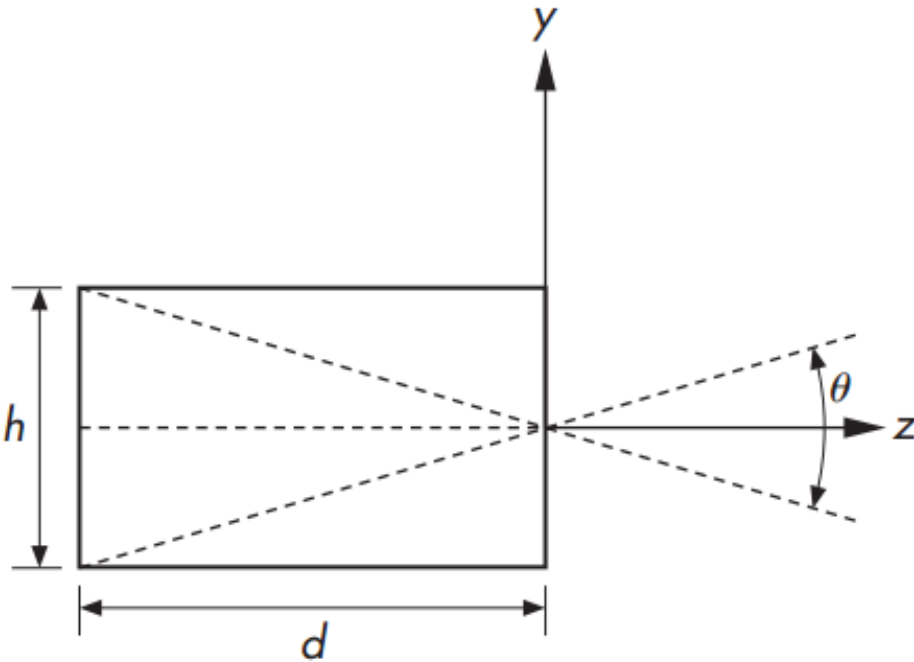
- Suppose that we orient our camera along the +z axis, with the pinhole at the origin of our coordinate system.
- We assume that the hole is so small that only a single ray of light, emanating from a point, can enter it.
- The film plane is located at a distance d from the pinhole.

We make use of trigonometry to find **projection** of point at (x,y,z)

$$x_p = -x/z/d \quad y_p = -y/z/d \quad z_p = d$$

These are equations of simple perspective

Pinhole Camera



By replacing the pinhole with a lens, we solve the two problems of the pinhole camera:

1] the lens gathers more light than can pass through the pinhole: The larger the **aperture** of the lens, the more light the lens can collect.

2] by picking a lens with the proper focal length **f** (a selection equivalent to choosing **d** for the pinhole camera) we can achieve any desired angle of view (up to 180 degrees).

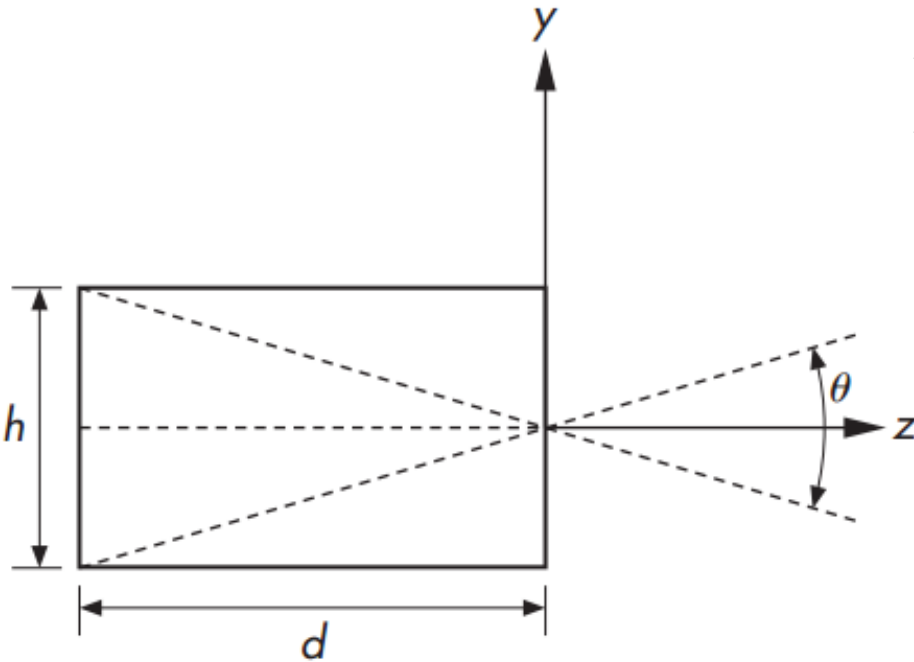
Lenses, however, do not have an infinite depth of field: Not all distances from the lens are in focus.

θ is called **field of view (FOV or angle of view)** :
the angle made by the largest object that the camera can image on its film plane.

$$\theta = 2 \tan^{-1} \frac{h}{2d}$$

Pinhole Camera

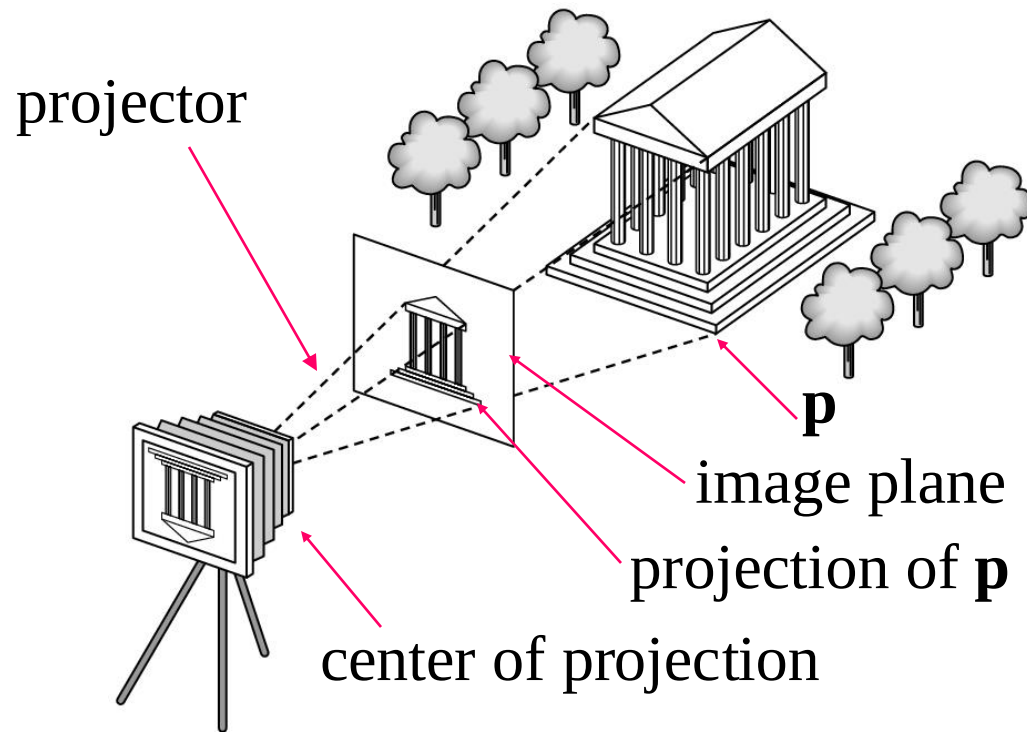
For example, for 35 mm film which is 36 mm wide and 24 mm high, $h=36$ mm would be used to obtain the horizontal field of view and $h=24$ mm for the vertical angle.



$$\theta = 2 \tan^{-1} \frac{h}{2d}$$

θ is called **field of view (FOV or angle of view)** :
the angle made by the largest object that the camera
can image on its film plane.

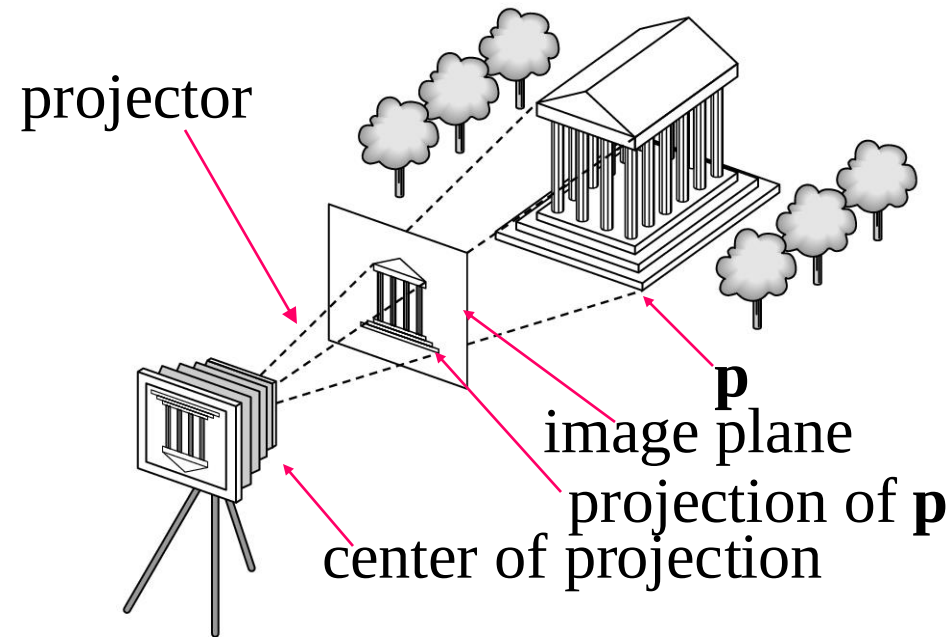
Synthetic Camera Model



Synthetic Camera Model

23

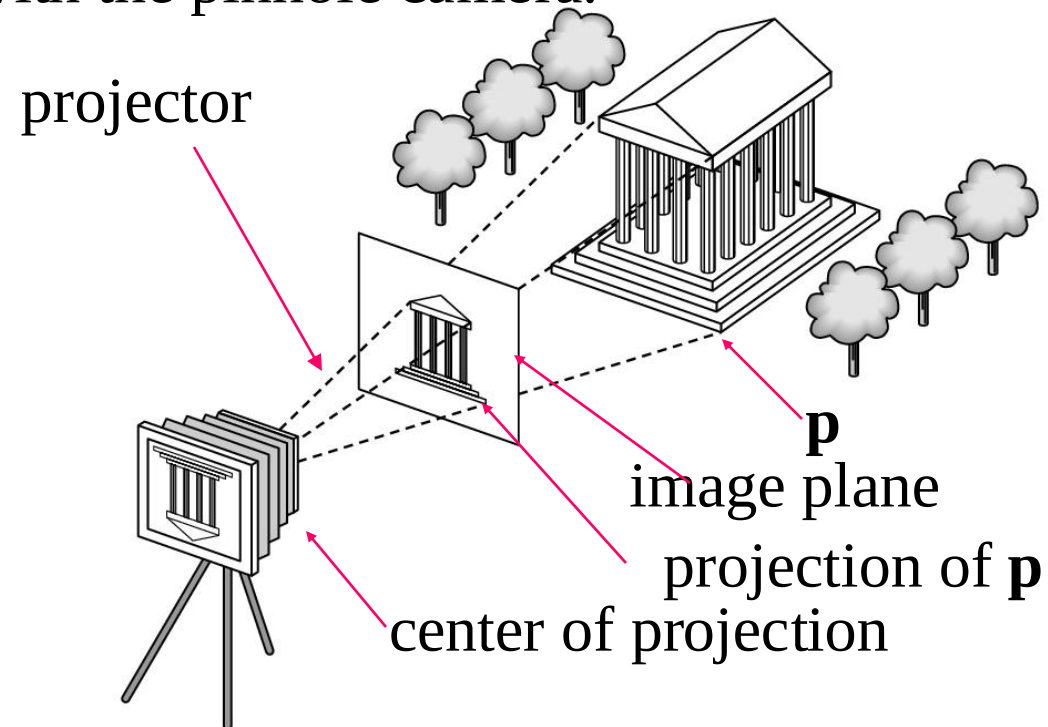
- We look at creating a computer generated image as being similar to forming an image using an optical system.
- This paradigm has become known as the synthetic-camera model.
- In the optical system, the image is formed on the film plane at the back of the camera.
- We need to identify a few basic principles, so that we can emulate this process to create artificial images.



Synthetic Camera Model

24

1. First, the specification of the objects is independent of the specification of the viewer.
 - Hence, we should expect that, within a graphics library, there will be separate functions for specifying scene objects and the viewer(camera).
2. Second, we can compute the image using simple geometric calculations, just as we did with the pinhole camera.

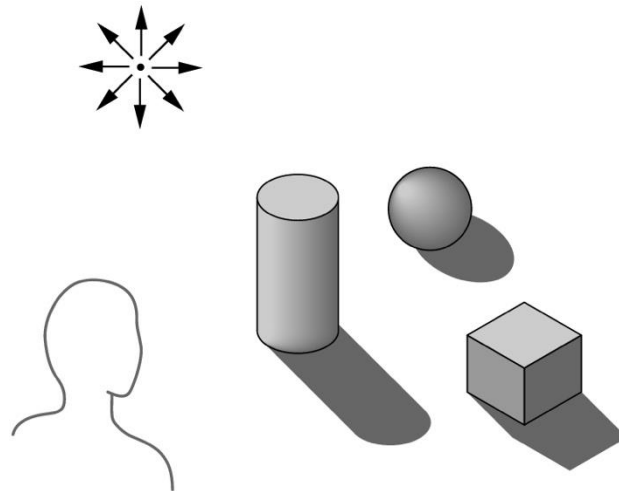


Advantages

- Separation of objects, viewer, light sources
- Two-dimensional graphics is a special case of three-dimensional graphics
- Leads to simple software API
 - Specify objects, lights, camera, attributes
 - Let implementation(system) determine image
- Leads to fast hardware implementation

Global vs Local Lighting

- Cannot compute color / shade of each object independently
 - Some objects are blocked from light
 - Light can reflect from object to object
 - Some objects might be translucent



Why not ray tracing?

- Ray tracing seems more physically based
- Actually simple for simple objects such as polygons and quadrics with simple point sources
- In principle, can produce global lighting effects such as shadows and multiple reflections but ray tracing is computationally heavier

Physical Approaches

- **Ray tracing:** follow rays of light from center of projection until they either are absorbed by objects or go off to infinity
 - Can handle global effects
 - Multiple reflections
 - Translucent objects
 - **Computationally heavy**
 - Must have **whole database available** at all times
- **Radiosity:** Energy based approach
 - Very slow

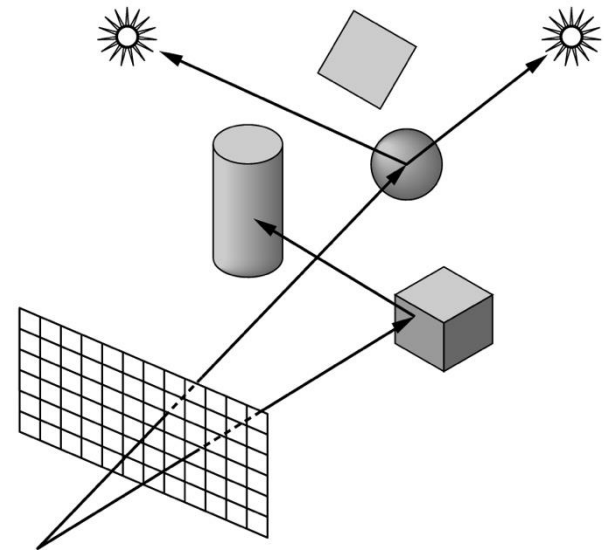
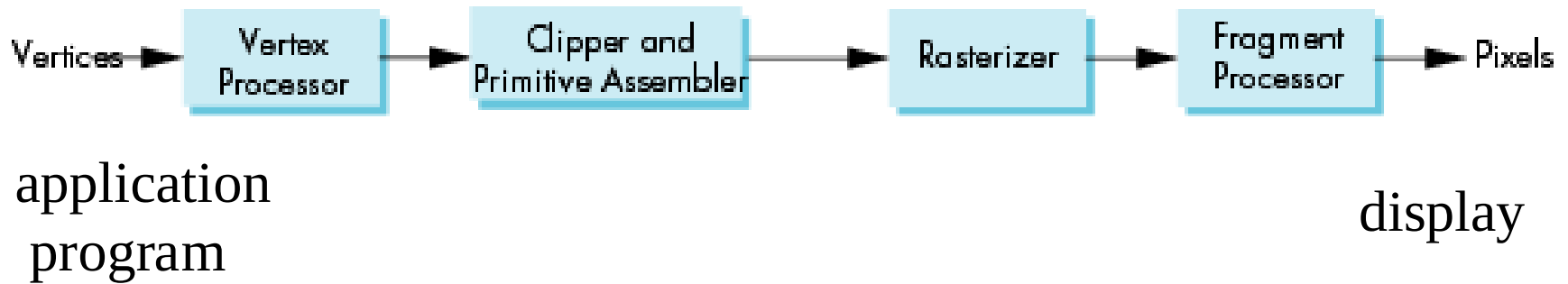


Image Formation Revisited

- Can we mimic the synthetic camera model to design graphics hardware software? Yes
- Application Programmer Interface (API)
 - **Need only specify**
 - Objects
 - Materials
 - Viewer
 - Lights
- But how is the API implemented?

Practical Approach

- Process objects one at a time in the order they are generated by the application
 - Can consider only **local lighting**
- Pipeline architecture :



- All steps can be implemented in hardware on the graphics card

Vertex Processing

- Much of the work in the pipeline is in converting object representations from one coordinate system to another
 - Object coordinates
 - Camera (eye) coordinates
 - Screen coordinates
- Every change of coordinates is equivalent to a matrix transformation
- Vertex processor can also compute per-vertex color



Projection

- *Projection* is the process that combines 3D viewer with 3D objects to produce a 2D image
 1. **Perspective projections:** all projectors meet at the center of projection
 2. **Parallel projections:** projectors are parallel, center of projection is replaced by a direction of projection



Primitive Assembly

Vertices must be collected into **geometric objects** such as

- Line segments
- Polygons
- Curves and surfaces

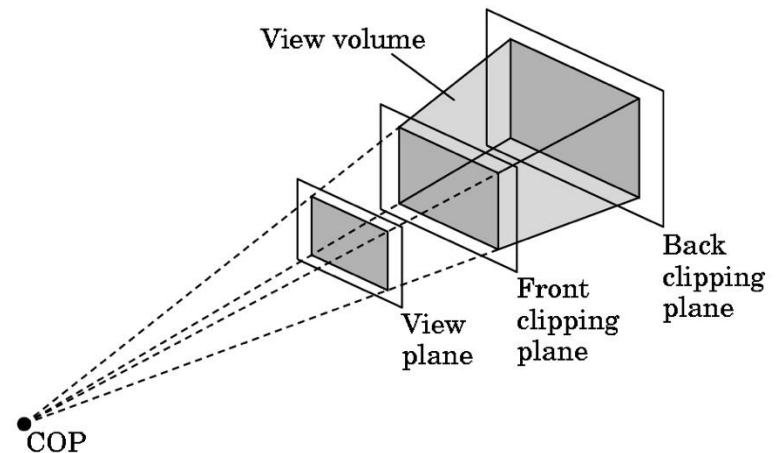
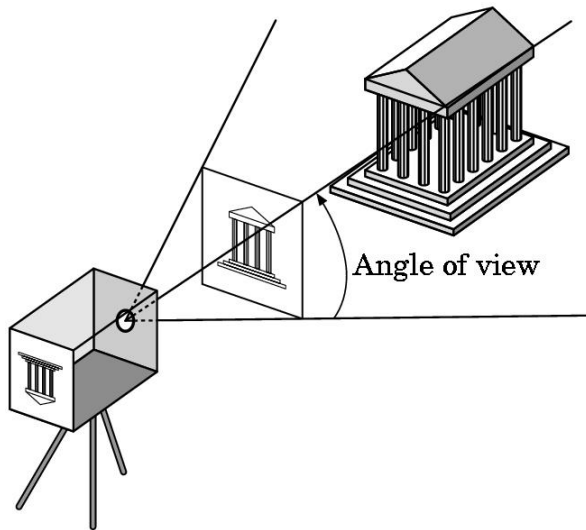
before clipping and rasterization can take place



Clipping

Just as a real camera cannot “see” the whole world out there, the virtual camera can only see a portion of the world or object space. This portion is called the **view volume**.

- Parts that are not within this volume are said to be **clipped out** of the scene



Rasterization

- If an object is not clipped out, the pixels corresponding to that object in the frame buffer must be assigned colors
- Rasterizer produces a set of fragments for each object
- Fragments are “potential pixels”
 - Each has a location in frame buffer
 - Also color and depth attributes
- Vertex attributes are interpolated over objects by the rasterizer



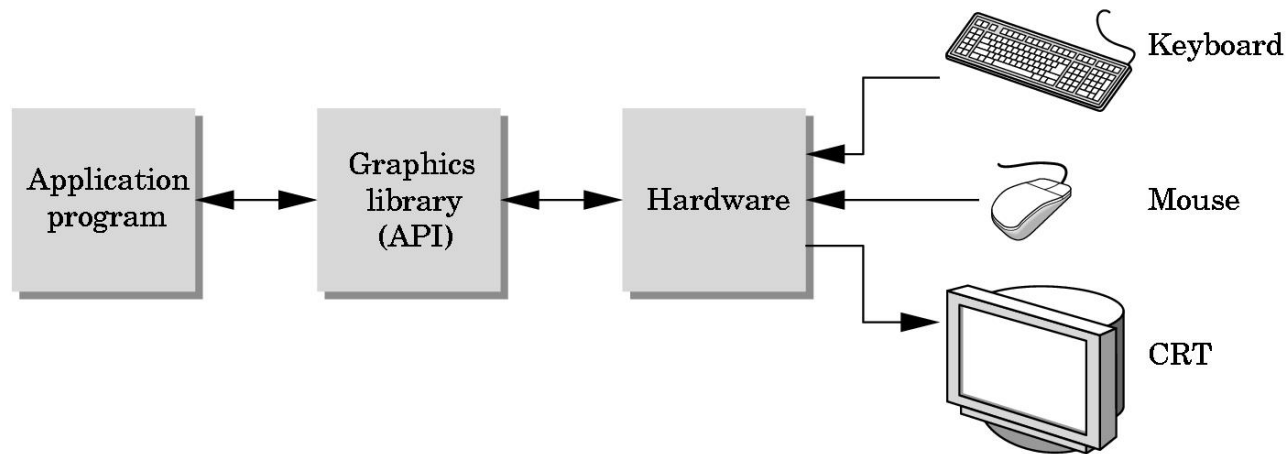
Fragment Processing

- Fragments are processed to determine the final color of the corresponding pixel in the frame buffer
- Colors can be determined by various ways including texture mapping and interpolation of vertex colors
- Fragments may be blocked by other fragments closer to the camera
 - **Hidden-surface removal**



The Programmer's Interface

- Programmer sees the graphics system through a software interface: the Application Programmer Interface (API)

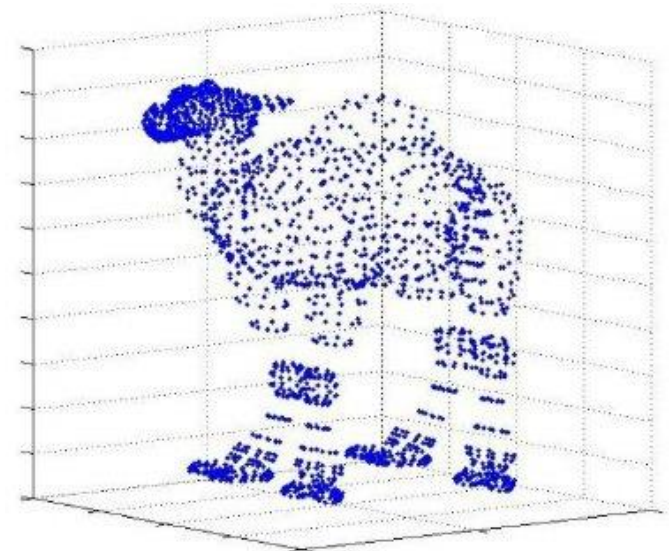


API Contents

- Functions that specify what we need to form an image
 - Objects
 - Viewer
 - Light Source(s)
 - Materials
- Other information
 - Input from devices such as mouse and keyboard
 - Capabilities of system

Object Specification

- Most APIs support a limited set of primitives such as
 - Points (0D object)
 - Line segments (1D objects)
 - Polygons (2D objects)
 - Some curves and surfaces
 - Quadrics
 - Parametric polynomials
- All are defined through locations in space: *vertices*



Example (GPU based)

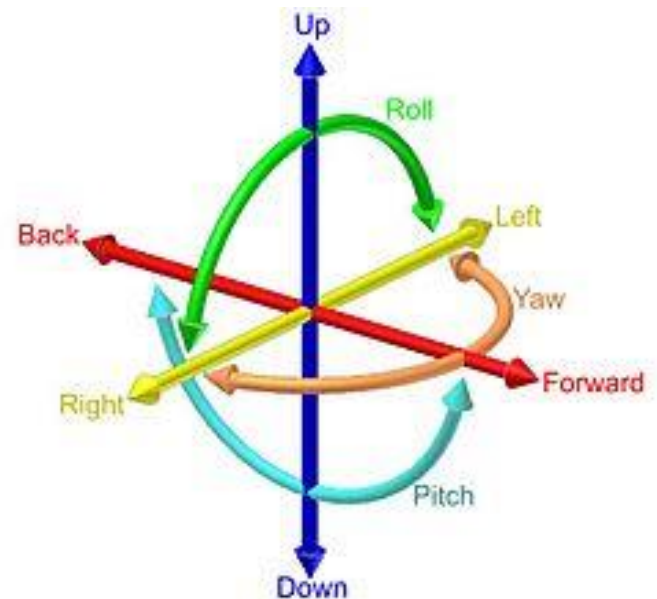
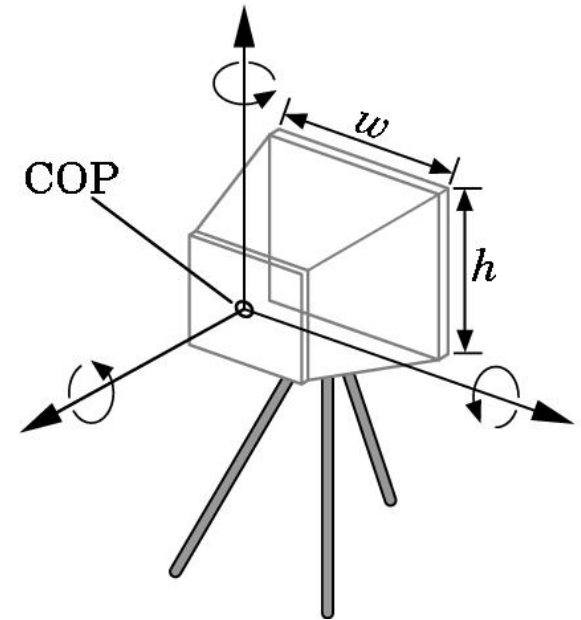
- Put geometric data in an array

```
vec3 points[3];  
points[0] = vec3(0.0, 0.0, 0.0);  
points[1] = vec3(0.0, 1.0, 0.0);  
points[2] = vec3(0.0, 0.0, 1.0);
```

- Send array to GPU
- Tell GPU to render as **triangle**

Camera Specification

- Six degrees of freedom
 - Position of center of lens
 - Orientation



Lights and Materials Specification

- Types of lights
 - Point sources vs distributed sources
 - Spot lights
 - Near and far sources
 - Color properties
- Material properties
 - Absorption: color properties
 - Scattering
 - Diffuse
 - Specular